

Konzeption und Umsetzung eines 2D-Metroidvania-Plattformers in Godot 4.7

Autor	Romed Kelmer	**Umfang**	6 Level inkl. Bosskampf
Lehrveranstaltung	Game & XR Experience	**Repository**	https://github.com/MCIkero/Glimm
Engine / Sprache	Godot Engine 4.7 (Forward+), GDScript	**Stand**	Oktober 2026

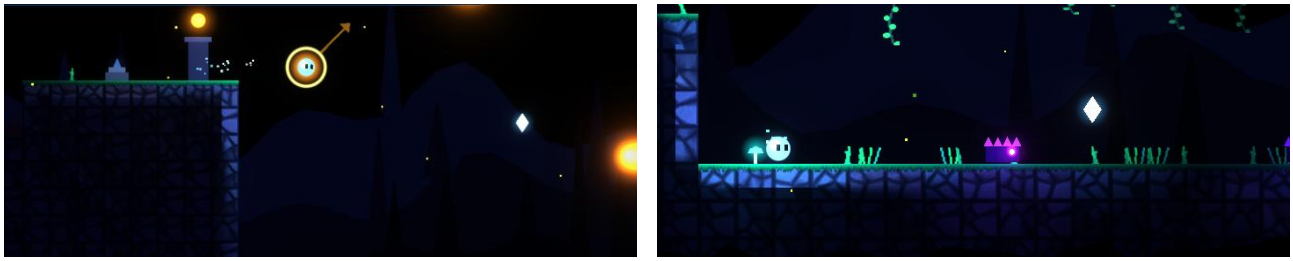


Abbildung 1: Links hält sich Glimm an einem Lichtsamen fest, die Zeit läuft langsamer und ein Pfeil zeigt die Flugrichtung. Rechts Level 3 mit Krabblern, Leuchtpilz und Lichtsplitter. Glimm ist die einzige Lichtquelle und wirft dynamische Schatten.

1 Game Design Sheet

1.1 Spielidee

Glimm ist ein kleiner Lichtgeist, der sich durch dunkle Höhlen bis zum *Lichtbaum* am Ende jedes Levels durchschlägt. Inspiriert hat mich ***Ori and the Will of the Wisps*** (Moon Studios, 2020). Dort hat mir vor allem das flüssige Movement gefallen, also Doppelsprung, Walljump und Dash, und ganz besonders der „Bash“: Man hält sich an einem Objekt fest, die Zeit verlangsamt sich, man zielt und schleudert sich davon. Diese Ideen habe ich aufgegriffen, Grafiken, Figuren und Namen aus Ori aber bewusst nicht.

> **Kernidee**
> In Glimm ist Licht **Sichtweite, Lebensenergie und Ressource** in einem. Je weniger Licht Glimm hat, desto kleiner wird der Bereich, den man sieht. Weil jede Fähigkeit Licht kostet, muss man ständig abwägen: schnell vorankommen oder lieber Licht für das nächste Lichttor sparen?

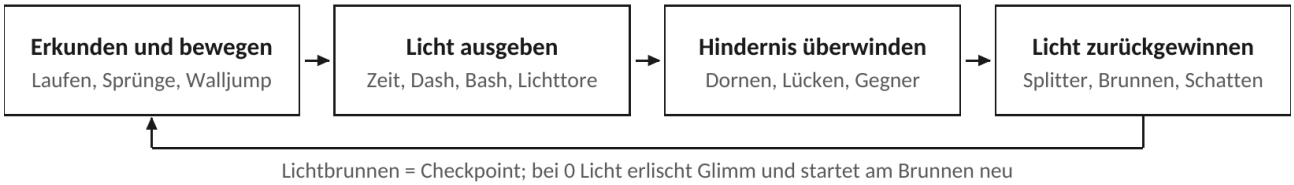
−1,2/s	**−20**	**−10**	**−40/50**	**+35**	**+15/20**	**100**
Licht verfällt ständig	Dash	Bash	Lichttor öffnen	Lichtsplitter	Schatten besiegt	Lichtbrunnen

Tabelle 1: Lichtökonomie in Lichtpunkten (Maximum 100). Fällt das Licht auf 0, erlischt Glimm und startet am letzten Lichtbrunnen neu. Je weniger Licht übrig ist, desto weiter zieht sich die Vignette am Bildrand zu.

1.2 Zielgruppe

Glimm richtet sich an Spieler:innen ab etwa 12 Jahren, die präzise 2D-Plattformer wie *Ori*, *Celeste* oder *Hollow Knight* mögen. Die Level sind kurz, Hinweistexte führen nach und nach an die Mechaniken heran, und gespielt wird mit Tastatur oder Controller (Xbox und PlayStation).

1.3 Core Gameplay Loop



Langfristige Schleife: Ein Schrein schaltet eine Fähigkeit frei, dadurch werden neue Wege erreichbar (Metroidvania).

Abbildung 2: Im Kleinen gibt man Licht aus und holt es sich zurück. Im Großen schalten neue Fähigkeiten Wege frei, die vorher nicht erreichbar waren (Metroidvania-Prinzip).

1.4 Mechaniken, Gegner und Level

Element	Beschreibung
Bewegung	Variable Sprunghöhe, Coyote Time, Jump Buffer, Doppelsprung, Walljump (auch an einer einzelnen Wand)
Dash	Kurzer Sprint ohne Schwerkraft; mit Sprung abbrechbar (Dash-Jump behält Schwung); besiegt Krabbler
Bash	An Lichtsamen, Irrlicht oder Projektil festhalten → Zeitlupe → zielen → wegschleudern; frischt Doppelsprung und Dash auf
Schatten-Irrlicht	Schwebt oder patrouilliert, Berührung ist tödlich – aber bash-bar
Krabbler	Läuft am Boden, dreht an Kanten und Wänden um; nur per Dash besiegbar
Speier	Schießt bei Sichtkontakt Kugeln; eine zurückgebashte Kugel zerstört ihn
Schattenherz (Boss)	Schwebt über der Arena, 6 Leben, 3 Phasen: gezielte Kugel → Dreier-Fächer + Krabbler → Kugelring + Dunkelheit. Nur mit eigenen, zurückgebashten Kugeln besiegbar
Level	1 Wurzelhöhle (Tutorial, Dash/Bash) · 2 Der Schacht (vertikal) · 3 Schattenhain (Gegner-Parcours) · 4 Irrlichtschlucht (Bash-Ketten) · 5 Dunkle Tiefe (dunkler, schnellere Irrlichter) · 6 Herz der Schatten (alles kombiniert + Bosskampf). Steigende Torkosten und Irrlicht-Tempo, weniger Licht

1.5 Steuerung und Feedback

Aktion	Tastatur	Controller
Laufen / Zielen	A/D bzw. ←/→ · W/S zum Zielen	Linker Stick / D-Pad
Springen (halten = höher)	Leertaste oder J	A / Kreuz
Dash	Shift oder K	X / RB
Bash (halten, zielen, loslassen)	E oder L	Y / LB
Pause · Neustart	Esc/P · R	Start · Back

Die Hinweistexte zeigen immer die Tasten des Geräts, das zuletzt benutzt wurde. Damit sich das Spiel gut anfühlt, steckt viel Arbeit im Feedback: Squash & Stretch, Partikel, Soundeffekte, Screenshake, Zeitlupe beim Bash und ein kurzer Farbversatz bei Bash und Tod. Wird das Licht knapp, flackert die Lichtleiste, und der Lichtkegel um Glimm schrumpft mit der Lichtmenge.

2 Technische Architektur

2.1 Technologie

Umgesetzt habe ich das Spiel in **Godot 4.7** mit statisch typisiertem **GDScript** (48 Skripte, rund 2.900 Zeilen, 33 Szenen). Gerendert wird mit **Forward+** (Vulkan) und HDR für 2D, weil Glow sonst nicht möglich wäre. Mir war wichtig, Godots Node- und Szenensystem konsequent zu nutzen. Alles, was man sieht, besteht aus Nodes (Sprite2D, Polygon2D, Line2D,

Label) und wird mit dem AnimationPlayer animiert, die Skripte enthalten nur Logik. Kommuniziert wird über Signals, und globale Zustände gibt es nur in Autoloads.

2.2 Szenenstruktur

```
MainMenu (Control) ──► Main (Node)     Autoloads: InputHints · Events · GameState · Audio
Main
├─ WorldEnvironment          Glow / Farbanpassung
├─ Level  (aus resources/level_catalog.tres geladen)
│    ├─ Darkness (CanvasModulate) · Background (Parallax2D + Polygon2D)
│    ├─ Terrain (TileMapLayer, Terrain-Set „Fels“) · TerrainShadows (LightOccluder2D-Nodes)
│    └─ Hazards · Objects · Decor (Gras/Ranken/Pilze) · Enemies · Hints · Fireflies
├─ Player (CharacterBody2D)
│    ├─ Visual (Sprite2D/Polygon2D) · Effects (Partikel + Sounds) · Light · Camera
│    ├─ WallCheckLeft/Right (RayCast2D) · BashDetector (Area2D)
│    └─ StateMachine — Idle · Run · Jump · Fall · WallSlide · Dash · Bash · Dead
├─ Foreground · PostFx       CanvasLayer (Silhouetten, Vignetten-Shader)
└─ HUD · PauseMenu           CanvasLayer (UI, läuft auch pausiert)
```

2.3 Zentrale Klassen

Klasse / Datei	Aufgabe
Player	Gemeinsame Bausteine: Schwerkraft, Laufen, Timer, Fähigkeits-Checks; alle Werte als @export im Inspector justierbar
StateMachine, PlayerState	Endlicher Automat; jeder Zustand ist ein eigener Node. Basisklassen GroundedState und AirborneState vermeiden doppelten Code
GameState (Autoload)	Licht, Fähigkeiten, Checkpoint, Zeit, Tode; meldet Änderungen per Signal
LevelCatalog, LevelEntry	Eigene Resource-Klassen: Levelliste als .tres, im Inspector per Drag-and-drop bearbeitbar
Audio (Autoload-Szene)	Musik mit Überblendung, UI-Sounds, Low-Pass-Effekt auf dem Music-Bus im Pausenmenü
Events (Autoload)	Event-Bus für einmalige Ereignisse (Tod, Hinweis, Levelende, Screenshake)
InputHints (Autoload)	Erkennt Tastatur/Controller und ersetzt Platzhalter wie {jump} in Texten
BashOrb → ShadowWisp, ShadowBolt	Bash-Ziel als Basisklasse; Gegner erben die Bash-Logik und überschreiben nur Aussehen und Verhalten
Crawler, Spitter	Bodengegner (CharacterBody2D + RayCast-Kantencheck) und Geschützturm (Sichtlinie per RayCast2D)
ShadowHeart, BossArena	Boss meldet nur phase_changed/defeated; die Arena steuert Tor, Krabbler, Dunkelheit und Musik, das HUD hört über den Event-Bus zu. Treffer über die gemeinsame Methode take_reflected_hit()
Level, Hud, PauseMenu, PostFx	Level stellt Spawn und Weltgrenzen bereit; UI und Effekte reagieren nur auf Signale
AutoTerrain, TerrainShadows, Decor	@tool-Skripte im Editor: korrigieren Kachel-Varianten, erzeugen Schatten-Occluder beim Malen und verteilen Deko per Inspector-Button (siehe 2.5)

2.4 Entkopplung über Signals

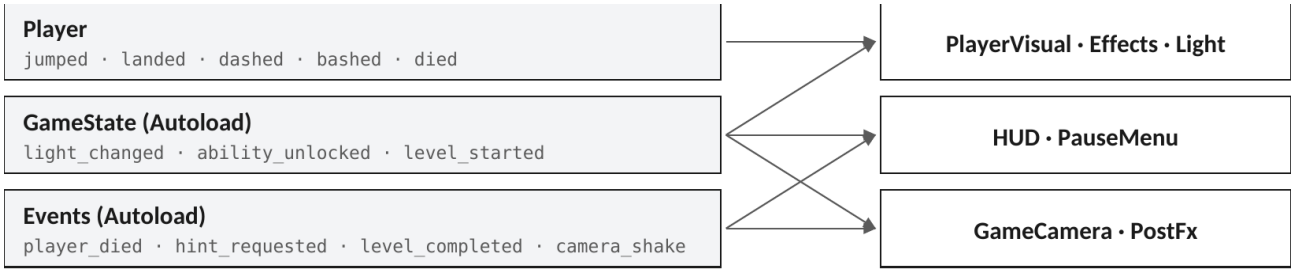


Abbildung 3: Signalfluss. Die Sender wissen nicht, wer zuhört, sie senden nur ihr Signal.

Ein Beispiel: Stirbt Glimm, wechselt die Zustandsmaschine in *Dead*. Der Player sendet *died*, worauf Visual und Partikel reagieren. Gleichzeitig meldet der Event-Bus *player_died*, das HUD blendet ab, PostFx erzeugt einen Farbversatz und die Kamera wackelt. Im Player-Code kommt keiner dieser Empfänger vor.

2.5 Alles im Editor: Level, Optik und Sound

Jedes Level ist eine normale Godot-Szene aus einheitlichen Felsquadraten mit 32 × 32 px. Den Fels male ich mit dem TileMap-Werkzeug, das Terrain-Set „Fels“ wählt dabei selbst die passende Moos-, Wurzel- oder Steinkachel. Objekte und Gegner ziehe ich als Szenen ins Level und stelle sie im Inspector ein (Schwebestrecke, Dornenbreite, Torkosten, Fähigkeit), @tool-Skripte zeigen diese Werte direkt im Editor an. Die Schatten sind gespeicherte *LightOccluder2D*-Nodes, die sich beim Malen neu aufbauen. Gras, Ranken und Leuchtpilze verteilt ein Inspector-Button (*@export_tool_button*) als einzelne Szenen-Instanzen, die ich danach frei verschieben kann. Für kleine Bauten wie Treppe und Brücke am Anfang von Level 1 gibt es außerdem einen einzelnen Felsblock (block.tscn), den man wie einen Bauklotz setzt. Neue Level entstehen aus level_template.tscn und kommen danach in den Level-Katalog.

Für den Glow sorgen *WorldEnvironment* und HDR-Farben über 1,0. Dazu kommen dynamische Schatten, Normal Maps über *CanvasTexture* und Shader für Wind (Gras, Ranken), Lichttore und Post-Processing. Beim Sound hat jede Szene ihre eigenen *AudioStreamPlayer*-Nodes. Der Sprung nutzt zum Beispiel einen *AudioStreamRandomizer*, damit er nicht jedes Mal gleich klingt. Musik und Effekte laufen über getrennte Audio-Busse.

2.6 So startet man das Spiel

Anleitung

1. Godot 4.7 in der Standard-Version (nicht .NET) von godotengine.org herunterladen und starten.
2. *Importieren* wählen und die Datei project.godot im Projektordner öffnen.
3. Mit **F5** starten und im Hauptmenü „Spielen“ wählen oder direkt ein Level aus der Liste.
4. Läuft Vulkan auf einem sehr alten Rechner nicht: Projekteinstellungen → Rendering → Renderer auf „Mobile“ stellen.

3 Learnings & Challenges

3.1 Was gut funktioniert hat

- **Zustandsmaschine:** Dash, Bash und Walljump konnte ich als neue Zustände ergänzen, ohne bestehenden Code umzubauen. Wenn etwas nicht funktioniert hat, wusste ich sofort, in welchem Zustand ich suchen muss.
- **Signals und Event-Bus:** HUD, Pausenmenü und Post-Effekte sind erst spät dazugekommen. Am Spieler-Code musste ich dafür nichts ändern.
- **Vererbung bei Gegnern:** Irrlicht und Projektil erben von BashOrb und waren dadurch sofort bash-bar. Das Zurückschleudern von Projektilen, das ich aus Ori kannte, hat so nur wenige Zeilen gekostet.
- **Editor-Workflow:** Weil alles aus Nodes, Szenen und Resources besteht, kann ich Level, Aussehen, Animationen und Sounds ändern, ohne Code anzufassen. Knifflige Abstände, etwa eine Lücke, die nur mit Dash zu schaffen ist, habe ich zusätzlich mit automatischen Testläufen geprüft, die Eingaben simulieren.
- **Licht als roter Faden:** Gegner geben Licht zurück, Tore verlangen Licht und die Vignette zeigt, wie viel noch übrig ist. Dadurch greift alles ineinander und das Konzept wirkt rund.

3.2 Hürden und Lösungen

Problem	Ursache	Lösung
Kachel-Varianten aktualisierten sich beim Malen nicht	Das Signal changed von TileMapLayer kommt bei Zellenänderungen nicht zuverlässig	Virtuelle Methode _update_cells() überschrieben; Update gesammelt per call_deferred
Bash-Halte-Limit zu lang	Engine.time_scale verlangsamt auch delta	Echtzeit = delta / time_scale; UI-Tweens ignorieren time_scale
Felsen komplett schwarz	Kachel-Occluder beschatten die Kachel selbst; flache Lichtstrahlen werden an Kachelkanten blockiert	TerrainShadows erzeugt zusammengefasste Occluder, an freien Seiten 20 px eingerückt

Problem	Ursache	Lösung
Krabbler zitterte an Kanten und fiel hinunter	Nach dem Umdrehen lieferte der Kanten-RayCast noch das Ergebnis der alten Seite	RayCast sofort versetzen und force_raycast_update() aufrufen
Boss wurde nie getroffen	Zurückgebashte Kugeln trafen Decke und Plattformen	Reflektierte Kugeln kollidieren nur mit Layer „enemies“ und lenken leicht zum Ziel (Gruppe reflect_target)
Fehler „flushing queries“ in Phase 2	Krabbler wurden im Physik-Callback des Treffers erzeugt	Erzeugung per call_deferred() nach dem Physik-Schritt

3.3 Kritische Reflexion

- **Audio:** Die Soundeffekte sind synthetisch erzeugt und klingen eher schlicht. Lautstärken und Abmischung habe ich nicht mit Testpersonen abgestimmt.
- **Balancing und Playtests:** Lichtverbrauch, Torkosten und Sprungabstände habe ich durchgerechnet und mit Testläufen geprüft, andere Leute haben das Spiel aber kaum gespielt. Wie schwer es wirklich ist und wo Frust entsteht, weiß ich deshalb noch nicht genau.
- **Lesbarkeit und Dunkelheit:** Das Konzept lebt von Dunkelheit, das Gameplay braucht aber Lesbarkeit. Gegner und Gefahren musste ich deshalb nachträglich aufhellen. Mit einem Farbkonzept von Anfang an hätte ich mir das sparen können.
- **Technische Grenzen:** Die Lichtwerte liegen noch in GameState statt in eigenen Resources, „Deko verteilen“ überschreibt von Hand gesetzte Deko, Forward+ setzt Vulkan voraus und Speicherstände gibt es noch nicht.

> **Ausblick**
> Als Nächstes würde ich bessere Soundeffekte und Speicherstände einbauen, Fähigkeits- und Gegnerdaten in Resources auslagern, weitere Bosse mit eigenen Mechaniken entwerfen und vor allem mit der Zielgruppe testen.

4 Quellenverzeichnis

Quelle	Verwendung	Lizenz
Godot Engine 4.7 – godotengine.org	Engine, Editor, Laufzeit	MIT
Open Sans (Godot-Standardschrift)	Schrift in HUD und Menüs	SIL Open Font License 1.1
Hintergrundmusik (background_sound_1/2.ogg)	Musik in Menü und Leveln	-
Soundeffekte	Selbst erzeugt (synthetisch generiert)	–
Grafik, Animationen, Shader, Level	Vollständig selbst erstellt; Kachel-Textur und Normal Map teilweise prozedural generiert	–
Ori and the Will of the Wisps (Moon Studios, 2020)	Inspiration für Spielgefühl, Movement und Bash-Mechanik, keine Assets, Figuren oder Namen übernommen	–
Claude (Anthropic), KI-Assistent	Unterstützung bei Code, Audio und Dokumentation	–